Session 2 — Paper exercises (solution)

Reference: Sutton & Barto, *Reinforcement Learning: An Introduction* (2nd ed.), Chapters 4 and 6. Python code for this gridworld (Shangtong Zhang). No computer needed.

The 4×4 gridworld (Example 4.1)

States are numbered as in the book. The two shaded corners are the same terminal state, written T:

T	1	2	3
4	5	6	7
8	9	10	11
12	13	14	T

- Actions: up, down, left, right. Moves are **deterministic**; an action that would leave the grid leaves the state unchanged.
- Reward: -1 for every transition, until the terminal state is reached.
- Policy: equiprobable random policy, $\pi(a \mid s) = 1/4$ for every action.
- No discounting: $\gamma = 1$.

Because the transitions are deterministic, action a in state s always leads to the same next state s'_a with the same reward r_a. The Bellman expectation equations become:

$$q_\pi(s,a) = r_a + \gamma\, v_\pi(s'_a), \qquad v_\pi(s) = \sum_a \pi(a \mid s)\, q_\pi(s,a) = \sum_a \pi(a \mid s)\big(r_a + \gamma\, v_\pi(s'_a)\big).$$

The value function of the random policy (from the lecture) is:

0	-14	-20	-22
-14	-18	-20	-20
-20	-20	-18	-14
-22	-20	-14	0

Part A — Values and policy improvement

A1 (Exercise 4.1). Compute $q_\pi(11, \text{down})$.
Answer: From 11, "down" reaches the terminal state with reward -1, and $v_\pi(T) = 0$: $q_\pi(11, \text{down}) = -1 + 0 = -1$.

A2 (Exercise 4.1). Compute $q_\pi(7, \text{down})$.
Answer: From 7, "down" reaches state 11 with reward -1: $q_\pi(7, \text{down}) = -1 + v_\pi(11) = -1 - 14 = -15$.

A3. Draw the **greedy policy** with respect to v_π: in each state, the action(s) leading to the neighbour with the highest value.

1. What do you notice about this policy?
2. The policy improvement theorem says that the greedy policy π' satisfies $v_{\pi'} \geq v_\pi$. Does the theorem guarantee that π' is *optimal*? What happens if we apply policy improvement again?

Answer:

T	$\leftarrow$	$\leftarrow$	$\leftarrow\downarrow$
$\uparrow$	$\uparrow\leftarrow$	$\leftarrow\downarrow$	$\downarrow$
$\uparrow$	$\uparrow\rightarrow$	$\rightarrow\downarrow$	$\downarrow$
$\uparrow\rightarrow$	$\rightarrow$	$\rightarrow$	T

(1) It always moves along a shortest path to the terminal state: it is already the **optimal** policy, after a single improvement step. (2) No: the theorem only guarantees $\pi' \geq \pi$. Here we are lucky, because the values of the random policy are ordered like the distances to the terminal state. In general, policy iteration needs several evaluation/improvement steps. If we improve again, nothing changes: π' is greedy with respect to its own value function, so it satisfies the Bellman optimality equation, which is the stopping condition of policy iteration.

Part B — Connecting mathematics and code

The following line comes from an implementation of iterative policy evaluation for this gridworld (`state_values` holds the current estimate V):

```
value += ACTION_PROB * (reward + discount * state_values[next_i, next_j])
```

B1. Which part of the Bellman expectation equation does this line implement? Why is there no sum over s'?
Answer: It adds one term $\pi(a \mid s)\left(r_a + \gamma V(s'_a)\right)$ of the sum over actions; the loop over the four actions computes the whole sum. There is no sum over s' because the transitions are deterministic: each action has a single next state (in general we would also sum over s', weighted by $\mathcal{P}^a_{ss'}$).

Part C — The AB example (Sutton & Barto, Example 6.4)

We observe 8 episodes in a Markov reward process with two states A and B, no discounting:

episode 1:	A, 0, B, 0	
episodes 2–7:	B, 1	(six times)
episode 8:	B, 0	

("A, 0, B, 0" means: start in A, reward 0, move to B, reward 0, end.)

C1. What estimates $V(A)$ and $V(B)$ does **Monte Carlo** give (average of the observed returns)?
Answer: Monte Carlo sets $V(s)$ to the average of the returns observed from s, where a return is the total reward from s to the end of the episode.

- $V(B)$: B appears once in each of the 8 episodes. The returns from B are 0 (episode 1), 1 six times (episodes 2–7) and 0 (episode 8), so $V(B) = (0 + 6 \times 1 + 0)/8 = 0.75$.
- $V(A)$: A appears only in episode 1, and the return from A is $0 + 0 = 0$. The average of this single return is $V(A) = 0$.

C2. Now replay these 8 episodes over and over with **TD(0)** until the values stop changing. What are $V(A)$ and $V(B)$? *Hint: TD uses the update target* $R + V(S')$.
Answer: TD(0) moves $V(s)$ towards its target $R + V(S')$. Replaying the episodes repeats these updates until the values stop changing.

- $V(B)$: the episode always ends after B, and a terminal state has value 0, so the target is just the observed reward: 1 in six episodes and 0 in two. $V(B)$ settles at the average of these targets, $(6 \times 1 + 2 \times 0)/8 = 0.75$, as with MC.
- $V(A)$: A always gives reward 0 and moves to B, so its target is $0 + V(B) = 0.75$. Each replay moves $V(A)$ towards this target, for example from 0 with $\alpha = 0.1$ once $V(B)$ has settled: $0 \to 0.075 \to 0.143 \to 0.203 \to \ldots$ It stops changing when it equals its target, so $V(A) = 0.75$.

The methods differ on $V(A)$: MC uses the single return observed from A, while TD uses where A leads, namely B, whose value is estimated from all 8 episodes.

C3. Which answer is "right"? Explain what each method is actually computing.
Answer: Both are right for different criteria. MC minimises the squared error on the observed returns (best fit to the data: the only return from A was 0). Batch TD(0) gives the exact values of the most likely Markov model of the data (A goes to B with probability 1 and reward 0; B ends with reward 1 with probability 3/4): this is *certainty equivalence*. If the process really is Markov, TD's answer should generalise better to future data; if the states are not Markov, MC is safer.

Final reflection

In **one sentence**, explain the difference between dynamic programming and the learning methods of today (MC, TD).
Answer: DP computes exact expectations using the known model $\mathcal{P}, \mathcal{R}$ (full-width backups), whereas MC and TD estimate the same quantities from sampled transitions, without a model (sample backups).